

INTERVIEW PREP · PHONE EDITION

Technical Interview Prep

Full Stack Developer — Redspher

TypeScript · React · Node.js · AWS · REST · AI / LLM /
MCP / Agents · CI/CD · PHP legacy

Prepared for Richard · July 2026

Contern, Luxembourg · Logistics / on-demand transport

Optimized for phone reading — narrow pages, large
type.

Contents

- 1 · The role, decoded 3
- 2 · Your pitch & proof map 5
- 3 · TypeScript 8
- 4 · React 14
- 5 · Node.js 20
- 6 · REST API design 26
- 7 · AWS 30
- 8 · AI: LLMs, MCP, agents 36
- 9 · Git, CI/CD, Docker 45
- 10 · The PHP legacy angle 47
- 11 · System design, logistics-flavored 50
- 12 · Behavioral & business questions 53
- 13 · Questions to ask them 56
- 14 · Company snapshot 57
- 15 · Cheat sheet & final checklist 58

How to use: skim the Q&A cards, say the answers out loud, star anything shaky. Green boxes = your personal talk tracks. Amber boxes = honesty guardrails. Blue boxes = story skeletons to fill in.

1 · The role, decoded

What the job ad is actually telling you.

Redspher wants a **product-minded full-stack developer**, not a pure specialist. Reading between the lines of the ad:

- **"Business-oriented development team" + "pragmatic mindset"** — they will probe how you translate vague business needs into shippable software and how you make trade-offs. Expect behavioral questions to carry real weight.
- **"Strong TypeScript and React"** — this is the hard bar. Expect the deepest technical drilling here, likely with live coding or a code review exercise.
- **"Backend services" + Node.js + REST + AWS** — solid mid-to-senior backend competence expected, but the ad leads with frontend, so backend questions will likely be practical (API design, queues, data) rather than deep distributed-systems theory. You can exceed the bar here — that's your edge.
- **AI: "MCP, AI agents, orchestration", "explore and prototype"** — unusual to name MCP explicitly in a job ad. They want someone who can prototype AI features responsibly. Very few candidates have real MCP experience; **you run your own MCP server**. Lead with it.
- **"PHP ... legacy systems, is a plus"** — they have a legacy PHP estate and a migration story in progress. Your early-career PHP plus a clear stran-

gler-fig narrative turns a "plus" into a differentiator.

- **"International environment"** — Luxembourg company, many nationalities; your years at Docter Luxembourg answer this directly.

Likely interview shape

Typical for this profile: (1) HR/culture screen, (2) technical interview covering TS/React/Node/AWS + an AI conversation, (3) live coding or take-home (React component or small TS API), (4) team/manager fit round on business collaboration. Prep for all four; this doc covers 2–4.

2 · Your pitch & proof map

Your 60-second intro, and evidence for every line of the ad.

60-second self-intro (say it out loud until smooth)

"I'm a senior full-stack engineer with 16 years in software, most recently at Docler Holding here in Luxembourg, where I held both Senior Frontend and Senior Backend Engineer roles. On the frontend I worked professionally with React; on the backend I owned Node.js services at serious scale — including a real-time event distribution system serving around 50,000 concurrent clients, and a full end-to-end rebuild of a customer-support chat that cut its infrastructure cost by roughly 75%. I'm double AWS-certified — Solutions Architect and Developer Associate — with production Lambda and S3 work. And for the last stretch I've gone deep on AI-assisted development: I use AI agents daily, I've given internal talks on AI and neural networks, and I built and operate my own MCP server that exposes a document-generation pipeline as tools for LLM clients. That mix — TypeScript across the stack, AWS, and hands-on MCP/agent work — is exactly why this role caught my eye."

Requirement → your proof

They ask for	You bring
Full-stack experience	16 years; distinct Senior Frontend and Senior Backend roles at Docler
	Professional React years at Docler (not self-study); TS across current projects

Strong
TypeScript/React

Node.js backend	High-traffic broadcaster (~50k concurrent, ~100k msg/s out); chat rebuild (Redis pub/sub WebSocket, -75% infra cost, -30% p95)
AWS	SAA + DVA certified; production Lambda + S3 at Docler; rest of serverless stack from certification labs
REST APIs	Years of API work; Distributed Checkout reference project (NestJS, outbox, saga, OIDC)
AI / LLM / MCP / agents	Own MCP server in production use (prompt-cv pipeline); daily agentic-AI workflow; internal AI talks; spec-driven agent development
Git, CI/CD	GitHub Actions + Helm + Argo CD Git-Ops in personal platform; professional Git throughout career
PHP legacy (plus)	PHP from the WebGarden era; comfortable maintaining and integrating legacy while migrating to TS
Business-driven, international	Cost/latency outcomes you can quote; years in Luxembourg's multicultural tech scene

Honesty guardrails — do not cross these

- **Two separate stories, never merged:** the chat rebuild (Redis pub/sub, low traffic, -75% cost, -30% p95, full ownership) and the event broadcaster (~50k concurrent, ~100k msg/s,

partial redesign, ~10% delivery speedup). Mixing their numbers will unravel under follow-ups.

- **AWS:** Lambda + S3 are real production work. API Gateway, DynamoDB, SQS/SNS, EventBridge, Step Functions, CloudWatch are *certification + hands-on labs* — say exactly that, confidently: "certified on it, practiced in labs, haven't run it in production yet."
- **Never claim** production Terraform, OpenTelemetry, DDD, C4, or ADRs. If asked: "I know the concepts, I haven't practiced them in production."
- **SQL/Oracle** = university plus ongoing self-study, not production DBA-level. Frame as "solid working SQL, actively sharpening it."
- **Kubernetes/IaC** knowledge comes from DVA certification, New Horizons training, and your k3s/Argo CD personal platform — a great honest story on its own.

Why honesty wins here

"Pragmatic" teams smell inflation instantly. A crisp "I haven't run that in production, but here's how I'd approach it" answered well scores *higher* than a bluff — it shows exactly the self-aware, business-safe judgment they're hiring for.

3 · TypeScript

The hard bar. Aim for crisp, senior-level answers with one concrete example each.

Q. Why TypeScript on a business product, beyond "fewer bugs"?

Types are executable documentation and a refactoring safety net: renaming a field or changing an API contract becomes a compiler-guided task instead of a grep-and-pray. They encode business rules (`OrderStatus` as a union, not a string) so invalid states don't compile. Crucially, types are **erased at runtime** — so at every boundary (HTTP, queue, env vars) you still validate with something like Zod and infer the static type from the schema, one source of truth.

Q. interface vs type ?

Mostly interchangeable for object shapes. `interface` supports declaration merging and is idiomatic for public/extendable contracts; `type` is required for unions, intersections, mapped and conditional types, and tuples. My rule: interfaces for object contracts, type aliases for everything else — and above all, one consistent team convention.

Q. any VS unknown VS never ?

`any` switches the compiler off in both directions — it infects everything it touches. `unknown` is the safe top type: you can assign anything to it, but must narrow before use — the right type for external input. `never` is the empty type: unreachable code, exhaustive-check leftovers. In strict codebases, `any` should appear only at deliberately quarantined boundaries.

Q. Show me generics with a constraint.

```
function pluck<T, K extends keyof T>(
  items: T[], key: K
): T[K][] {
  return items.map(i => i[key]);
}
// pluck(orders, "status") -> OrderStatus[]
```

`K extends keyof T` ties the key to the object's real properties, so typos fail at compile time and the return type is inferred precisely.

Q. Which utility types do you actually use?

`Partial` / `Required` for patch payloads, `Pick` / `Omit` to derive DTOs from a domain model, `Record<K,V>` for keyed maps, `ReturnType` / `Parameters` when wrapping functions, `Awaited` for unwrapping promises, `Readonly` for immutable views. The principle: derive types from one canonical shape instead of hand-copying near-duplicates that drift apart.

Q. Discriminated unions — why do seniors love them?

```
type ShipmentEvent =  
  | { type: "picked_up"; at: string }  
  | { type: "delayed"; at: string; reason:  
    string }  
  | { type: "delivered"; at: string; pod:  
    string };  
  
switch (e.type) {  
  case "picked_up": ...  
  case "delayed":    ...  
  case "delivered": ...  
  default: { const x: never = e; return x; }  
}
```

The `type` tag narrows automatically in each branch, and the `never` default makes the switch **exhaustive**: add a new event variant and every unhandled switch becomes a compile error. Perfect fit for logistics event streams.

Q. Type guards and narrowing?

Built-in narrowing via `typeof`, `instanceof`, `in`, and discriminant checks. Custom user-defined guards return `value is X`; assertion functions (`asserts value is X`) throw instead of returning false — good for invariants. Caveat: a wrong hand-written guard lies to the compiler, so for external data I prefer schema validation (Zod's `safeParse`) which narrows *and* actually checks.

Q. `satisfies` vs `as`?

`as` is an assertion — it overrides the compiler and can hide real errors. `satisfies` checks a value against a type *while preserving the narrower inferred type*: `const config = {...} satisfies AppConfig` validates the shape but keeps literal types for autocompletion. Use `satisfies` for config objects and lookup tables; reserve `as` for the rare boundary you genuinely know better than the compiler (plus `as const` for literal freezing).

Q. Which strict flags matter to you?

`strict: true` as the baseline — above all `strictNullChecks`, which turns the billion-dollar mistake into a compile error. Beyond it I value `noUncheckedIndexedAccess` (array/record lookups return `T | undefined`, forcing you to handle the miss) and `exactOptionalPropertyTypes`. Mentioning these two signals you've run strict TS in anger, not just enabled defaults.

Q. TypeScript is structurally typed — implications?

Compatibility is by shape, not by name: any object with the right members fits. Great for flexibility; risky when two shapes coincide by accident (`CustomerId` vs `OrderId` , both strings). Mitigations: excess-property checks on object literals, and **branded types** (`type OrderId = string & { __brand: "OrderId" }`) when mixing IDs would be a real business bug.

Q. How would you migrate a JS (or legacy-adjacent) codebase to TS?

Incrementally: enable `allowJs` + `checkJs` , convert file-by-file starting with the most-shared modules, add `.d.ts` for untyped internals, ratchet strictness up (start loose, tighten per-flag), and gate CI on "no new any ". Never a big-bang rewrite — the codebase must stay shippable throughout. The same ratchet mindset applies to wrapping their legacy PHP behind typed API contracts.

Q. How do you keep API types in sync between frontend and backend?

One source of truth, generated outward: either Zod schemas shared from a common package (infer both runtime validation and static types), or an OpenAPI spec with generated clients/types. What I avoid is hand-maintained duplicate interfaces on both sides — they always drift, and the compiler can't see across the HTTP gap.

4 · React

Expect the deepest drilling here. Anchor answers in the rendering model.

Q. Walk me through React's rendering model.

State or props change → React re-renders that component and, by default, its whole subtree → produces a new element tree → reconciliation diffs it against the previous one → only real differences are committed to the DOM. Keys give list items stable identity across renders so state and DOM nodes are reused instead of destroyed. Two consequences: render functions must be pure (they may run more than once), and performance work means either moving state down or memoizing boundaries — not fighting the DOM.

Q. `useEffect` — how do you use it correctly?

Effects synchronize the component with an *external* system: subscriptions, timers, imperative APIs. Rules I follow: exhaustive deps (with the lint rule on), always return cleanup for anything subscribed, and treat StrictMode's double mount-unmount in dev as a free test that cleanup works. What effects are **not** for: derived state (compute during render), reacting to user events (do it in the handler), or chaining state updates through cascading effects — that's the classic spaghetti source.

Q. When do `useMemo` / `useCallback` / `React.memo` actually help?

They exist for referential equality: `React.memo` skips re-render if props are shallow-equal, so callbacks/objects passed into memoized children need `useCallback` / `useMemo` to keep stable identities. Also for genuinely expensive computations and for stabilizing effect dependencies. But memoization is not free — extra memory and comparison work, and premature sprinkling hides real structural problems. My order: measure with the Profiler, restructure state placement first, memoize the proven hot boundaries last.

Q. How do you manage state in a serious React app?

First split **server state** from **client state**. Server state (orders, quotes, shipments) belongs in a data-fetching cache like TanStack Query: caching, deduplication, background refetch, invalidation, optimistic updates — solved problems you shouldn't rebuild in Redux. Client/UI state stays local as long as possible; when it must be shared, something light like Zustand, or Redux Toolkit where the team already standardized on it. Context is for low-frequency globals (theme, auth identity), not a state manager.

Q. What's the pitfall with Context?

Every consumer re-renders whenever the context value changes — and an inline `value={{...}}` object changes every render. Mitigations: memoize the value, split one fat context into several narrow ones (state vs dispatch), or use a subscription-based store with selectors when updates are frequent. Context distributes; it doesn't optimize.

Q. Controlled vs uncontrolled inputs — and forms in general?

Controlled: React state is the source of truth (`value` + `onChange`) — full control, but every key-stroke renders. Uncontrolled: the DOM holds the value, read via ref. For real business forms I reach for react-hook-form: uncontrolled under the hood so it's fast, with schema validation (Zod resolver) so form rules and API contracts share one definition.

Q. Code splitting and error handling in the UI?

Route-level splitting with `React.lazy` + `Suspense` keeps the initial bundle honest; heavy widgets (charts, maps) split separately. Error boundaries wrap routes/feature shells so one crashing widget degrades to a fallback instead of white-screening the app; pair them with reporting so failures are seen, not swallowed.

Q. What did React 18/19 change that you actually use?

React 18: automatic batching everywhere, `useTransition` / `useDeferredValue` to keep typing responsive while expensive lists re-render, `useId`, and Suspense maturing. React 19: form Actions with `useActionState` / `useOptimistic` reduce hand-rolled pending/error state, `use()` for reading promises, `ref` as a normal prop, and Server Components going mainstream via frameworks like Next.js — I'd frame RSC as "aware and have used in my own Next.js App Router project, pragmatic about adopting where the team's framework supports it."

Q. A page is slow. Your debugging workflow?

Reproduce → React DevTools Profiler to see what renders and why (which props/state changed) → typical culprits: state living too high, unstable object/callback props defeating memo, huge lists without virtualization, layout-thrash from effects. Fix structurally (move state down, split components, virtualize with `react-window`), memoize proven hot paths, then re-measure. For load performance: bundle analysis, code splitting, and caching headers before micro-optimizations.

Q. Common React anti-patterns you'd flag in review?

- Array index as key on reorderable/filterable lists — state bleeds between items.
- Copying props into state ("mirror state") — instant staleness; derive instead.
- Effect chains updating state that triggers more effects — restructure the data flow.
- One 800-line component owning fetch + form + table — split by responsibility.
- Business logic buried in JSX — extract hooks/pure functions so it's testable.

Q. How do you test React?

React Testing Library with the user's perspective: query by role/label, interact, assert visible outcomes — not implementation details, so refactors don't shred the suite. Network mocked at the boundary with MSW so components run their real fetch paths. Unit-test extracted hooks and pure logic directly; keep a thin layer of E2E (Playwright) for the money paths like checkout or quote submission.

Q. TypeScript in React — patterns you rely on?

Typed props with discriminated unions for variant components (`{ kind: "link"; href }` vs `{ kind: "button"; onClick }` — invalid combinations won't compile), generic components for tables/selects (`<Table<Order>>`), `ComponentProps<typeof X>` to extend existing components, and inferring server-state types from the shared API schema rather than redeclaring them.

5 · Node.js

You can outscore the bar here. Land the event loop cold, then bring your scale stories.

Q. Explain the event loop. (The classic — nail it.)

Node runs JS on a single thread; libuv drives an event loop with phases — **timers** (setTimeout/setInterval), pending callbacks, **poll** (I/O), **check** (setImmediate), close callbacks. Between callbacks, **microtasks** run to completion: `process.nextTick` first, then resolved-promise callbacks. I/O itself is delegated to the OS or libuv's thread pool (fs, DNS, crypto), so "single-threaded" only describes your JS. Consequence: any CPU-heavy synchronous work freezes every request — which is why you offload it.

Q. So how do you scale Node and handle CPU-bound work?

Per-process: keep handlers non-blocking; move CPU work (PDF generation, crypto, parsing huge files) to `worker_threads` or a separate service/queue. Per-machine: one process per core via cluster/PM2 — though in a containerized world the idiom is one process per container and horizontal scaling behind a load balancer with health checks. Sticky sessions only if you must (better: externalize state to Redis).

Q. Streams and backpressure — when do they matter?

Whenever data is larger than memory or arrives over time: exporting a 2M-row CSV, proxying file uploads to S3, transforming logs. Streams process chunks with bounded memory; **backpressure** is the flow control — a slow consumer pauses the producer via `pipe`/`pipeline` semantics instead of buffering unbounded. I use `stream.pipeline` (or web streams) because it propagates errors and cleanup correctly, which manual `.pipe()` chains famously don't.

Q. Error handling strategy in a Node service?

Async/await with try/catch at meaningful boundaries; a central error middleware/filter that maps known error classes to HTTP problem responses and everything else to a logged 500 with a correlation ID. Distinguish operational errors (retryable, expected) from programmer errors (crash-worthy). Register `unhandledRejection`/`uncaughtException` to log and exit — a half-broken process is worse than a restarted one. And **graceful shutdown**: on SIGTERM stop accepting connections, drain in-flight requests, close DB pools, then exit — essential for zero-downtime deploys.

Q. Express vs Fastify vs NestJS — your take?

Express: minimal, huge ecosystem, but everything is convention-by-discipline. Fastify: faster, schema-first validation and serialization built in — my default for lean services. NestJS: opinionated structure (modules, DI, guards, interceptors) that pays off when a team and code-base grow — it's what I chose for my Distributed Checkout reference project precisely to demonstrate production structure: typed config validation, TypeORM with migrations-only, OIDC guards, transactional outbox. The honest framing: the framework matters less than the boundaries you enforce with it.

Q. Configuration and secrets?

Twelve-factor: config from the environment, never committed; a schema (Zod) validates all env vars at boot so a missing variable fails fast at startup, not at 3 a.m. in a request. Secrets come from the platform's secret store (AWS Secrets Manager / SSM, or K8s secrets), injected at runtime; local dev uses `.env` files that are git-ignored. Separate config per environment, same artifact promoted through them.

Q. Top API security concerns you actually act on?

- Validate every input at the edge (schema validation) — injection and type-confusion die here.
- AuthN $\neq$ AuthZ: verify the JWT, *then* check this user may touch this resource (IDOR is the #1 real-world API hole).
- Rate limiting and payload size limits; helmet-style security headers.
- Parameterized queries / ORM — never string-built SQL.
- Dependency hygiene: lockfiles, `npm audit` / Dependabot in CI, minimal base images.
- No secrets or PII in logs — GDPR territory, especially with customer shipment data.

Q. What does observability look like in your services?

Structured JSON logs (pino) with a request/correlation ID propagated through every hop — that ID is how you reconstruct a shipment's story across services. Metrics for the four golden signals (latency, traffic, errors, saturation) with alerts on symptoms users feel (p95, error rate), not on CPU vanity. Liveness/readiness endpoints so the platform can restart and route around unhealthy instances. I know the concepts behind distributed tracing; I haven't operated OpenTelemetry in production and I'd say so plainly.

Q. A Node endpoint is slow. Walk me through it.

Measure first: is it the event loop (blocked — check event-loop lag), the downstream (DB/HTTP — check per-dependency timing from the request logs), or N+1 queries? Typical fixes in order of frequency: add the missing index / batch the N+1, add caching with explicit TTL and invalidation rules, pool and reuse connections, stream instead of buffering large payloads, offload CPU work. Profile with `clinic/0x` or the built-in inspector when it's genuinely CPU. Then lock the win in with a load test.

Your two scale stories — keep them separate and sharp

Story A — chat rebuild (ownership + business impact): "I rebuilt a customer-support chat end to end — architecture, backend, frontend, rollout.

WebSocket delivery with Redis pub/sub for cross-instance fan-out. Low traffic, but the old system was overbuilt: the rebuild cut infra cost ~75% and improved p95 latency ~30%. My favorite kind of win — simpler system, better numbers."

Story B — event broadcaster (scale): "Separately, I worked on a high-traffic event distribution system: ~50,000 concurrent clients, ~100,000 messages/sec outbound. I partially redesigned the delivery path and got ~10% faster delivery. That's where I learned to respect backpressure, fan-out cost, and measuring before touching anything."

If asked for details you don't want to overclaim on story B: "it was a partial redesign of an existing system — I can go deep on the parts I owned."

6 · REST API design

Logistics platforms live on APIs and webhooks. Idempotency is the star topic — bring it up yourself.

Q. How do you design a resource-oriented API?

Nouns for resources (`/shipments` , `/shipments/{id}/events`), HTTP verbs for actions, plural collections, sub-resources for ownership. Business operations that don't map cleanly to CRUD become explicit action resources (`POST /shipments/{id}/cancel`) — pragmatism over REST purity. Consistency beats cleverness: same naming, same pagination, same error shape everywhere.

Q. Status codes and error responses you standardize on?

200 read, 201 created (with Location), 202 accepted-async, 204 no body; 400 malformed, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict (state machine violations — "can't cancel a delivered shipment"), 422 semantic validation, 429 throttled; 5xx = our fault. Errors as a consistent envelope — RFC 7807 problem+json style with a machine-readable code, human message, and correlation ID. Never leak stack traces.

Q. Idempotency — explain it like it matters. (It does, here.)

An idempotent operation applied twice has the same effect as once. GET/PUT/DELETE are idempotent by contract; POST isn't — which is a problem, because networks retry. For "create order / book transport / request quote" I require an `Idempotency-Key` header: first request stores key + response; a retry with the same key returns the stored response instead of double-booking a truck. Same principle downstream: consumers of at-least-once queues and webhooks must deduplicate by event ID. In transport, a duplicate booking is real money — this is the answer that shows business-driven engineering.

Q. Pagination approach?

Cursor-based (opaque cursor from the last item's sort key) for anything large or hot: stable under inserts and $O(1)$ at any depth. Offset/limit only for small admin lists — page 50,000 with OFFSET is a table scan, and rows shift as data changes. Always return `next_cursor` and cap page size server-side.

Q. API versioning?

Additive-first: adding optional fields isn't a version bump, and clients must ignore unknown fields. For genuinely breaking changes, `/v2` in the path — visible, cacheable, debuggable — run in parallel with a deprecation window and usage metrics so you know when v1 can die. The deeper answer: versioning is a contract-management problem; OpenAPI + generated clients + contract tests catch breaks before customers do.

Q. AuthN/AuthZ for APIs?

OIDC/OAuth2 with an identity provider (I run Keycloak in my reference platform): authorization-code + PKCE for browser apps, client-credentials for service-to-service. Short-lived JWT access tokens — validated by signature, issuer, audience, expiry — plus refresh-token rotation. Claims carry identity and coarse roles; fine-grained authorization stays server-side against the resource ("is this shipment in this user's organization"), because tokens get stale and IDOR is the classic breach.

Q. Design webhooks between us and carrier partners.

Outbound: signed payloads (HMAC with a shared secret + timestamp to block replay), an event envelope with unique event ID + type + version, retries with exponential backoff and jitter until 2xx, a dead-letter list and a redelivery UI for partners. Inbound: verify signature, respond 2xx fast and process async via a queue, deduplicate by event ID because delivery is at-least-once. Document events like an API — they are one.

Q. When is REST the wrong tool?

Long-running work → 202 + status resource or job queue, not a 90-second request. Real-time push (tracking positions) → SSE or WebSocket. High-volume internal service-to-service where contracts and performance dominate → gRPC or async messaging. Aggregation-heavy UIs → a BFF layer so mobile isn't making seven calls. The senior answer is choosing per interaction pattern, not loyalty to one style.

7 · AWS

Double-certified — own that. Production truth: Lambda + S3. Everything else: "certified, labbed, ready to run it."

Q. Lambda vs Fargate vs EC2 — how do you choose?

Lambda: spiky or event-driven work, webhook handlers, glue — zero idle cost, scales to zero, but 15-min max, cold starts, and per-invocation pricing that inverts at sustained high load. **Fargate (ECS)**: long-lived services, WebSockets, steady traffic, or when you want containers without managing nodes. **EC2**: special hardware or maximum control, at the cost of patching and capacity management. Decision inputs: traffic shape, duration, latency sensitivity, and total cost at expected load — not fashion.

Q. Tell me about your production AWS work.

"At Docler I worked with Lambda and S3 in production — event-driven functions and object storage as part of real product workflows. Beyond that, I hold both the Solutions Architect and Developer Associate certs, and I've built out the wider serverless stack — API Gateway, DynamoDB, SQS/SNS, EventBridge, Step Functions, CloudWatch — in certification labs and personal projects rather than production. So I know the architecture and the sharp edges on paper and in practice-scale, and I ramp fast." (Exactly this framing — confident and precise.)

Q. DynamoDB or a relational DB?

DynamoDB when access patterns are known and key-addressable — session state, event streams, high-scale lookups: design the partition/sort key around the queries, GSIs for secondary patterns, on-demand capacity to start. Relational (RDS/Aurora Postgres) when you need ad-hoc queries, multi-entity transactions, reporting, constraints — which most business/logistics cores do. Honest default: Postgres for the system of record, DynamoDB/Redis for the hot paths that outgrow it. In Dynamo you model queries first, data second — the opposite of normalization instinct.

Q. SQS mechanics you should know cold?

At-least-once delivery → consumers must be idempotent. **Visibility timeout** hides an in-flight message; crash without deleting → it reappears — set timeout > worst-case processing, extend via heartbeat for long jobs. **DLQ** after N failed receives, with an alarm on DLQ depth — that's your poison-message safety net. Standard queues: huge throughput, occasional duplicates, loose ordering; FIFO: ordering + exactly-once-ish per message group, lower throughput. Batch receives for cost/perf.

Q. SNS vs EventBridge?

SNS: simple pub/sub fan-out — one event to many subscribers (email, Lambda, SQS), classic pattern SNS→SQS per consumer for durability. EventBridge: an event *bus* with content-based routing rules, schema registry, and native SaaS/AWS event sources — the choice when you're building event-driven architecture across services rather than just fanning out. Both decouple producers from consumers, which is the actual point.

Q. S3 patterns a full-stack dev uses weekly?

Presigned URLs — the browser uploads/downloads directly to S3 with a short-lived signed URL, so files never stream through your API (cheaper, faster, no payload limits). Event notifications (S3 → Lambda/SQS) to trigger processing on upload — POD photos, carrier documents. Lifecycle rules to tier old objects to cheaper storage. CloudFront in front for public assets. Block public access by default; access via IAM roles.

Q. IAM in one minute?

Identities get policies; services assume **roles** — no long-lived access keys baked into apps, ever. Least privilege: this Lambda can read this bucket prefix and write this queue, nothing more. Resource policies (bucket/queue side) complement identity policies; explicit deny wins. In practice: per-service roles defined in IaC, credentials injected by the platform, rotated automatically.

Q. Lambda operational sharp edges?

Cold starts (mitigate: small bundles via esbuild, provisioned concurrency for latency-critical paths); 15-minute cap; payload and /tmp limits; concurrency limits as both throttle risk and protective tool (reserved concurrency to shield a fragile downstream DB); init code outside the handler to reuse connections; DLQ/failure destinations for async invokes. And per-invocation logging in CloudWatch is your black box recorder — structure it.

Q. How would you do IaC and deploys to AWS?

Everything reproducible from code — for a TypeScript team, CDK is the natural fit: infrastructure in the same language, typed, code-reviewed, deployed through the pipeline with a diff step before apply. My hands-on IaC practice comes from certification training and my personal platform, where I run the equivalent flow with Helm charts and Argo CD GitOps on Kubernetes — same philosophy: git is the source of truth, the cluster converges to it, rollback is a revert.

Mini-architecture to sketch if asked: carrier webhook ingestion

API Gateway → thin Lambda: verify HMAC signature, drop invalid, enqueue raw event to **SQS** (respond 200 in <100 ms) → consumer (Lambda or Fargate) validates schema, **dedupes by event ID**, upserts into Postgres, emits a domain event on

EventBridge for downstream (notifications, analytics) → failures retry with backoff → poison messages land in a **DLQ** with an alarm. Idempotent at every hop, buffered against carrier bursts, and each stage independently scalable. Narrate the trade-offs out loud — that's what they're grading.

8 · AI: LLMs, MCP, agents

Your differentiator. Almost no candidate runs their own MCP server. Be the one who talks trade-offs, not hype.

Q. Explain how an LLM works — to a business stakeholder.

"It's a model trained on huge amounts of text to predict the next token, which in practice makes it very good at transforming and generating language: summarize this email thread, extract fields from this transport document, draft this customer reply. It only sees what's in its **context window**, it's probabilistic rather than deterministic, and it can state wrong things fluently. So we design around it: give it the right context, constrain its output, validate the result, and keep humans on the decisions that carry risk." That framing — capability plus guardrails — is what a business-oriented team wants to hear.

Q. What practical knobs and techniques do you use when integrating an LLM?

Clear system prompts with role, rules, and output contract; few-shot examples for format-sensitive tasks; low temperature for extraction/classification, higher only for creative drafting; **structured outputs** (JSON schema / tool schemas) so responses parse reliably — then validate with Zod anyway, because the schema constrains shape, not truth. Streaming to the UI for perceived latency. And prompt versioning in git: prompts are code.

Q. Explain RAG end to end.

Retrieval-Augmented Generation: instead of hoping the model "knows" your data, you fetch the relevant slices at question time and put them in the context. Pipeline: ingest documents → chunk sensibly (structure-aware, with overlap) → embed chunks into vectors → store in a vector index (pgvector fits nicely if you're already on Postgres) → at query time embed the question, retrieve top-k (ideally hybrid: vector + keyword, then rerank) → generate an answer grounded in those chunks, with citations. Wins: current data, access control at retrieval time, traceable answers. Failure modes: bad chunking, retrieval misses, and context stuffing — retrieval quality, not the model, is usually the bottleneck.

Q. RAG vs fine-tuning?

RAG for *knowledge* — fresh, per-tenant, auditable, cheap to update (re-index, not retrain).

Fine-tuning for *behavior and form* — consistent tone, domain phrasing, narrow structured tasks at volume. Most business cases start with prompting + RAG; fine-tune only when you have evals proving a persistent gap and enough quality data. Saying "I'd earn fine-tuning with eval evidence" is the pragmatic answer.

Q. How do you deal with hallucinations?

Layered: ground with retrieved context and instruct "answer only from the provided sources, else say you don't know"; constrain output to schemas; validate machine-actionable outputs against reality (does this shipment ID exist?); show citations so users can verify; keep a human approval step wherever the action has cost — sending a quote, changing a booking. And measure: an eval set of known Q&A pairs run on every prompt/model change, because vibes don't regress-test.

Q. How does tool/function calling actually work?

You expose functions to the model as name + description + JSON-schema parameters. The model doesn't execute anything — it returns a structured "call X with these args" message; *your code* validates the args, executes, and feeds the result back; the model continues with real data. Loop until it produces a final answer. Everything hinges on the schema and description quality, and on your runtime treating model output as untrusted input — validate before executing, always.

Q. What is MCP, and what's your hands-on experience?

"Model Context Protocol — an open standard, originated by Anthropic in late 2024 and since adopted broadly across the ecosystem, that standardizes how AI clients connect to tools and data. Instead of writing bespoke integrations per assistant, you run an MCP **server** exposing *tools*, *resources*, and *prompts* over JSON-RPC (stdio locally, streamable HTTP remotely), and any MCP-capable client can use it. **I built and operate my own**: a remote MCP server that exposes my document-generation pipeline — tools for tailoring and optimizing CVs — which I use from LLM clients daily. So I've dealt with the real questions: tool schema design, making tools idempotent and safe to retry, hosting, and what the model does with badly described tools." That last sentence is the credibility.

Q. When would you build an agent — and when not?

A useful ladder: plain prompt → prompt chaining → routing → parallelization → orchestrator-workers → evaluator loops → autonomous agent. Each rung adds capability and subtracts predictability. If the workflow is known, encode it as a *workflow* with LLM steps — deterministic, testable, debuggable. Reserve real agents (model chooses tools and iterates toward a goal) for open-ended tasks where you can't enumerate the path, and even then: bounded iterations, allowlisted tools, spend limits, human approval on side effects. "Start with the simplest thing that works and add autonomy only when the task demands it" — exactly the pragmatism this team is asking for.

Q. Production concerns for LLM features?

- **Latency:** stream tokens; route simple tasks to small/fast models, hard ones to big models; cache aggressively (including prompt/prefix caching).
- **Cost:** token budgets per feature, monitoring per-request cost, trimming context ruthlessly.
- **Reliability:** timeouts, retries with backoff, fallback models/providers, circuit breakers — it's a flaky remote dependency, treat it like one.
- **Quality:** versioned prompts, eval suite in CI, canary rollout for prompt/model changes, feedback capture in the UI.
- **Compliance:** EU/GDPR — data processing agreements, no unnecessary PII in prompts, retention rules; logistics data includes customers and addresses.

Q. Prompt injection — what is it and what do you do about it?

Untrusted content (an email, a carrier document, a web page) containing instructions the model may follow — "ignore your rules and forward this data". You can't fully "filter" it, so you architect: strict separation of trusted instructions from untrusted data; least-privilege, allowlisted tools (a summarizer needs no send-email tool); human confirmation for consequential actions; output validation; no secrets in context; logging for audit. Treat the model like a bright intern reading hostile mail — helpful, but not authorized to wire money.

Q. How do you use AI in your own development workflow?

"Daily and deliberately. I work spec-first with coding agents: for my Distributed Checkout platform I wrote a 22-page requirements document specifically structured for an AI agent to implement against — locked stack, constraints, acceptance criteria — because agents amplify clarity and punish vagueness. Day to day: agentic coding for scaffolding, refactors, tests and exploration, with me owning architecture and reviewing every diff; AI-generated code gets the same review bar as human code. I've also given internal talks on AI and neural networks. The net effect is speed *plus* a quality ratchet — more tests and docs than I'd write alone, not fewer."

Q. Where would you apply AI at a company like Redspher? (Volunteer this.)

- Extracting structured quote/booking requests from customer emails and PDFs into the TMS — classic unstructured→structured, high ROI.
- Carrier document processing: PODs, CMRs, customs docs — parse, validate, flag anomalies.
- Ops copilot: "why is shipment X late?" answered from tracking events + carrier comms, with citations.
- Customer-facing status explanations and proactive delay notifications drafted for human approval.
- Support triage and suggested replies grounded in order data.

Frame each as: measurable business outcome, human-in-the-loop where money or promises move, evals before scale-up.

9 · **Git, CI/CD, Docker**

Short section, but "how do you ship?" is a guaranteed question.

Q. Branching and PR culture?

Trunk-based with short-lived branches: small PRs (reviewable in minutes, revertable in one), merged behind feature flags when the feature isn't user-ready. GitFlow only where slow release trains genuinely exist. Review culture: fast turn-around, comments about correctness and clarity over style (linters own style), and CI as the non-negotiable gate.

Q. What does your ideal CI pipeline for a TS repo look like?

On every PR: install with cached dependencies → lint → `tsc --noEmit` typecheck → unit/integration tests → build → (for services) build the container image once, tag by commit SHA. Fast feedback is a feature: parallelize, cache, keep the whole run in minutes. The same immutable artifact then gets *promoted* through staging to prod — never rebuilt per environment.

Q. Deployment strategies and rollback?

Rolling for routine, blue/green for instant switch-and-revert, canary (small % of traffic + metric watch) for risky changes; feature flags decouple deploy from release so rollback is often a toggle, not a redeploy. The perennial gotcha is the database: expand-contract migrations — add nullable/new alongside, backfill, switch reads, remove later — so any app version in the window works with the schema and rollback stays safe.

Q. Docker for Node — the essentials?

Multi-stage build: deps + build in one stage, copy only production artifacts into a slim runtime image; run as non-root; respect signals so graceful shutdown works (exec-form CMD, handle SIGTERM); healthcheck endpoint; `.dockerignore`; pin base images. Small images deploy faster and carry fewer CVEs.

Your GitOps story

"On my personal platform I run the full loop: GitHub Actions builds and tests, Helm charts describe the deployment, and Argo CD keeps a k3s cluster converged to what's in git — deploy is a merge, rollback is a revert, and drift is visible. I built it deliberately to practice production-shaped delivery outside work hours." (All true, all yours.)

10 · **The PHP legacy angle**

Their "plus". Two minutes of prep turns it into a differentiator.

Q. What's your PHP background?

"PHP was part of my earlier career — I worked with it (alongside C#) at WebGarden. I wouldn't call myself a current PHP specialist, but I read and maintain it comfortably, and I've spent years around the real problem your ad hints at: keeping a legacy system earning money while you migrate around it. I'm very happy being the person who can open the legacy code, understand it, and wire it safely to the new TypeScript world."

Q. How do you migrate away from a legacy PHP system without stopping the business?

Strangler fig. First make the legacy observable and safe (logs, monitoring, tests around critical flows). Put a routing facade in front (gateway/proxy) so traffic can shift per-endpoint. Extract capability by capability into TS services, highest pain/value first; keep an anti-corruption layer translating legacy models to clean domain contracts so old quirks don't leak into new code. Data is the hard part: prefer clear ownership per capability with sync via events over long-term shared-database coupling. Retire endpoints as usage hits zero — measured, not assumed. Never big-bang.

Q. Practical PHP↔Node interop patterns?

REST or queue between the worlds with explicit contracts (OpenAPI even for internal); events out of PHP (outbox table polled or hooked) so new services react without synchronous coupling; idempotent consumers because retries happen; correlation IDs across both stacks so one trace tells the whole story. Avoid two writers to one table — that's where migrations go to die.

Q. Modern PHP awareness (in case they probe)?

Enough to be dangerous: Composer packages and autoloading, PSR standards, PHP 8.x niceties — typed properties, enums, attributes, match expressions, constructor promotion — and the Symfony/Laravel landscape. Honest line: "I can navigate a modern PHP codebase; my center of gravity is TypeScript."

11 · System design, logistics-flavored

Method first, always: clarify requirements → API → data model → scale path → failure modes → observability. Narrate trade-offs.

Scenario 1 — Real-time shipment tracking

Clarify: how many active shipments, update frequency, how fresh is "real-time" (2 s vs 2 min changes everything), web/mobile consumers.

Ingest: carrier/driver positions via API or webhook → validate → queue (burst buffer).

Store: latest position per shipment in Redis (hot read), append history to Postgres/timeseries for the trail.

Push: SSE or WebSocket to clients subscribed per shipment; SSE if it's one-way updates (simpler, proxy-friendly), WS if bidirectional. Fan-out via Redis pub/sub across instances.

Scale & failure: stateless socket tier scaled horizontally; clients reconnect with last-event-id; degrade to polling if push fails; dedupe out-of-order positions by timestamp.

Your hook: "This is home turf — my broadcaster work was exactly high-fan-out real-time delivery, and my chat rebuild used Redis pub/sub for cross-instance fan-out."

Scenario 2 — Instant quote / pricing service

Clarify: inputs (route, weight, vehicle, urgency), latency target (instant quote ⇒ $p95 < \sim 300$ ms?), quote validity window, audit needs.

Design: stateless quote API; pricing = rules/rate

tables + surcharges (fuel, distance) — cache reference data in memory/Redis with TTL + versioned invalidation; persist every issued quote with inputs + rate-version for audit and honoring.

Edge cases: reference-data update mid-flight (price with a consistent snapshot version), unavailable distance provider (fallback matrix, marked estimate), duplicate submissions (idempotency key).

Evolution: log quote→booking conversion — the data an ML pricing model later feeds on; expose the rules engine so business can tune without deploys. That last sentence *is* "business-oriented developer".

Scenario 3 — Carrier event/webhook ingestion at scale

Use the AWS mini-architecture from §7 (API GW → verify → SQS → idempotent consumer → Postgres → EventBridge → DLQ). Emphasize: respond fast + process async, dedupe by event ID, poison-message isolation, per-carrier rate isolation so one noisy partner can't starve the rest, replay ability from stored raw events.

Q. Monolith vs microservices — where do you stand?

"A well-modularized monolith is the right default for most teams — one deploy, one transaction boundary, refactor freely. I split a service out when there's a concrete forcing function: independent scaling (my broadcaster would be one), different runtime/failure isolation, or team ownership boundaries. Microservices trade code complexity for distributed-systems complexity — you pay in eventual consistency, sagas, and observability. I built my reference project around exactly those patterns (outbox, saga orchestration) to learn the cost consciously, which is why I don't romanticize it."

12 · Behavioral & business questions

"Business-driven and pragmatic" is in the ad — treat this section as equal weight to TypeScript. Answer in STAR: Situation, Task, Action, Result (with a number).

Flagship: "Tell me about a project you're proud of / drove end to end"

→ Chat rebuild. S: aging customer-support chat, overbuilt and costly. T: I owned the rebuild end to end — architecture, backend, frontend, rollout. A: simplified the design around WebSocket delivery with Redis pub/sub; migrated safely with fallback. R: ~75% lower infra cost, ~30% better p95, and a system one engineer can reason about. Moral: "the best engineering win is often subtraction."

"Working under constraints on a live system"

→ Event broadcaster. S: high-traffic event distribution, ~50k concurrent clients, ~100k msg/s out. T: improve delivery without breaking a system users depend on every second. A: measured first, re-designed part of the delivery path, changed incrementally behind metrics. R: ~10% faster delivery, zero incident rollout. Moral: "at scale, measurement is the first refactor."

"Learning something new fast" / "initiative"

→ Twin stories: (1) AWS: committed to a full-time cloud programme and both Associate certs to convert years of practice into architecture-level rigor. (2) MCP server: "MCP was brand new; I read the spec, built a remote server around a real pipeline of

mine, and now use it daily — I learn by shipping something I'll actually use."

"Disagreement with product / pushing back" — fill in your example

Skeleton: name the business goal you both shared → the cheaper/simpler path you proposed → how you made the trade-off visible (numbers, prototype, risk list) → outcome and relationship intact. If your best example is scope-cutting: "we shipped 80% of the value in 30% of the time and revisited the rest with real usage data." Pick one true story from your Docler years before the interview and rehearse it once out loud.

"A failure / production incident" — fill in yours

Pick a real one where *you* owned part of the miss. Structure: impact honestly stated → immediate mitigation → root cause (blameless, systemic) → the guardrail that now exists (test, alert, process). Interviewers grade the learning loop, not the absence of failure. Avoid any story you can't detail under three follow-up questions.

"International environment"

→ Direct: years at Docler Holding Luxembourg — multinational teams, English working language, cross-cultural collaboration as the daily normal, and you live in the region (Igel, on the Luxembourg border). For them this de-risks relocation/commute questions entirely.

Q. "Why Redspher?" — have a real answer.

Three honest threads to weave: (1) the tech match is unusually exact — TS/React/Node/AWS *plus* serious MCP/agent ambitions, which almost no local role offers; (2) logistics is a domain where software visibly moves physical outcomes — latency and correctness are trucks and money, which suits how you like to engineer; (3) Luxembourg roots: you've built your career there and want to build the next chapter locally rather than remotely. Say it in your own words; don't re-cite.

Q. "Why did you leave Docler / what happened since?"

Keep it forward-facing and unruffled: role ended in December; you've used the months deliberately — AWS certification track completed, built a production-grade reference platform (Distributed Checkout), shipped an MCP server, went deep on agentic development — and you're now choosing the right seat rather than the first seat. A calm gap-with-artifacts story reads as strength. Never badmouth the previous employer.

13 · Questions to ask them

Pick 3–4. They're also reconnaissance — the answers tell you what the job really is.

- "Where are you on the AI journey concretely — prototypes, or agents already touching production workflows? What's the next milestone?"
- "The ad mentions MCP specifically — are you building internal MCP servers around your logistics systems, or consuming existing ones?"
- "How does the team work with product — do engineers join discovery, or receive specs?" (Tests "business-oriented" from their side.)
- "What does the legacy landscape look like — how much PHP, and is there an active migration strategy?"
- "How often do you deploy, and what does the path to production look like — reviews, CI, environments?"
- "What does the testing culture look like today, honestly?"
- "How is the platform organized — per-brand systems (Easy4Pro, Upela, Flash...) or a shared core?"
- "What would make the person in this seat clearly successful after six months?"
- "Who decides architecture — is there a guild/architects, or do teams own their choices?"
- "What's the on-call / incident reality for this team?"

14 · **Company snapshot**

Enough context to sound prepared; verify anything you plan to quote.

- Luxembourg-based (HQ Contern) transport & logistics group focused on **on-demand, time-critical delivery**; formerly known as Flash Europe International, with roots back to 1981, rebranded Redspher around 2018.
- Operates as a platform of brands — including Flash, Easy4Pro, Easy2Go, Upela, Schwerdtfeger, SpeedPack Europe, Rubiwin — each specializing in urgent/time-sensitive transport niches.
- Order of magnitude: several hundred employees across Europe (plus Asia presence); revenue in the couple-hundred-million-USD range; private-equity backed.
- Strong verticals in automotive and healthcare/life-sciences urgent freight — domains where a late delivery stops a production line or worse; correctness and speed are the product.
- Current positioning leans hard on being a digital platform with **proprietary AI** and centralized tech powering the brands — which is exactly the wave this role rides. Connecting your MCP/agent experience to that narrative is the strongest single move you can make in the interview.

15 · Cheat sheet & final checklist

One-liners to have loaded

- **Event loop order:** sync code → `process.nextTick` → promise microtasks → timers → poll (I/O) → `setImmediate`.
- **Hooks rules:** top level only, same order every render; effects sync with external systems, cleanup is mandatory.
- **Memo trio:** `React.memo` skips child renders; `useCallback`/`useMemo` keep references stable so memo works.
- **Idempotency:** retries are inevitable ⇒ POST needs an Idempotency-Key; queue/webhook consumers dedupe by event ID.
- **Status codes:** 201 create · 202 async · 204 empty · 401 who? · 403 not yours · 409 state conflict · 422 invalid · 429 slow down.
- **DynamoDB:** design for access patterns; PK+SK; GSI for the second question; at-least-once everywhere ⇒ idempotent writes.
- **SQS:** visibility timeout, DLQ + alarm, FIFO = order per group at lower throughput.
- **MCP primitives:** server exposes tools / resources / prompts over JSON-RPC; stdio local, streamable HTTP remote.
- **Agent ladder:** prompt → chain → route → parallelize → orchestrate → evaluate → autonomous; climb only as far as the task forces you.

- **Migration mantra:** strangler fig, anti-corruption layer, expand-contract for data, measure then retire.

Last 24 hours

- Say the 60-second intro out loud 3x; once in the shower counts.
- Re-read the amber guardrails box (§2). Two stories, never merged.
- Rehearse: event loop, idempotency, MCP answer, "why Redspher".
- Pick and rehearse your real "pushback" and "failure" stories.
- Skim their brand sites (Easy4Pro, Upela, Flash) for 10 minutes — one concrete product observation beats generic praise.
- Prepare your 3–4 questions from §13; write them down.
- Environment check if remote: camera, mic, quiet, IDE ready for live coding (Node + TS + React scaffold warm).
- Sleep. A rested senior beats a crammed one.

Final frame

You're not asking for a favor — you're offering 16 years of shipped software, real scale stories, double AWS certification, and rare hands-on MCP/agent experience to a team that explicitly wants exactly that. Calm, concrete, honest. Good hunting.